4.10.1
Original clock cycle = Sum of all latencies in 4.7
= 250 + 150 + 25 + 200 + 150 + 5 + 30 + 20 + 50 + 50 = 930ps
Improved clock cycle= Sum of all latencies in 4.7 with register file increased by 10 ps
= 250 + 160 + 25 + 200 + 150 + 5 + 30 + 20 + 50 + 50 = 940ps

Now with LDUR and STUR being decreased by 12% and using 100 instructions to make percentages the actual number of instructions,
35 * 0.88 → 30.8 instructions for LDUR, STUR
Old instructions = 100, New instructions = 95.8

Speedup = (Old Clock-Cycle/Instructions) / (New Clock-Cycle/Instructions)
= (930/100) / (940/95.8) = 0.948
Speedup after the improvement is 0.948

4.10.2
Old Latency = (100 * 930 * 10^-12)^-1 = 10752688.172
New Latency = (96 * 940 * 10^-12)^-1 = 11081560.284

Change in latency = 11081560.284 - 10752688.172 = 328872.112 or .329 * 10^6
Latency Ratio = 11081560.284/10752688.172 = 1.031

Improvement Cost = 1000 + 2*200 + 10 + 100 + 30 + 2000 + 5 + 100 + 1 + 500 = 4146
Improvement clock cycle = 940 ps
Improvement cost/clock-cycle = 4146/940 = 4.411
Original cost = 1000 + 200 + 10 + 100 + 30 + 2000 + 5 + 100 + 1 + 500 = 3946
Original clock-cycle: 930ps
Original cost/clock-cycle = 3946/930 = 4.243

Cost change = 4.411 - 4.243 = 0.168
Cost ratio: 4.411/4.243 = 1.04

Change in latency = 329 * 10^6
Latency Ratio = 1.031
Cost change = 0.168
Cost ratio = 1.04

4.10.3
In the previous problem it was found that the cost and performance increased when the amount of registers are doubled. Therefore a situation where it would make sense to increase registers would be when performance is more important than cost and the increase in performance outweighs the increase in cost. It would also make sense when a greater amount of memory is accessed to use more registers as well.

A situation where it would not make sense would be when cost is more important then performance and therefore cannot be increased. Also, when the application does not necessarily need higher performance, then doubling the registers would not be sensical.

4.15.1
LDUR was the original critical path: 50 + 150 + 50 + 200 + 250 + 25 = 725 ps
LDUR and STUR do not need ALU and sign extend in the critical path, so now the critical path and clock cycle time is the instruction with the next biggest latency.
R-type: 50 + 2(150) + 200 + 25 = 575 ps
LDUR/ADD: 50 + 150 + 250 + 25 = 475 ps
CBZ: 50 + 150 + 50 + 200 + 150 + 25 = *625 ps*
B: 50 + 150 = 200ps
CBZ has the longest latency and is therefore the clock cycle time.
625ps

4.15.2
LDUR and STUR instructions now have their addresses calculated before being commanded, which is done by adding an extra ADD instruction before calling either LDUR or STUR.
Therefore, the LDUR/STUR-ADD latency = 475 + 525 = 1050 ps
LDUR and STUR instruction mix is doubled to 50% and 20% in instruction mix.
New Instruction Mix: 24% + 28% + 50% + 20% + 11% + 2% = 135%
New CPI:
((24/135)*575) + ((28/135)*575) + ((50/135)*1050) + ((20/135)*1050) + ((11/135)*625) + ((2/135)*200)) = 819.81 ps
Old CPI:
((0.24 * 575) + (0.28 * 575) + (0.25 + 725) + (0.10 * 700) + (0.11 * 625) + (0.02 * 200)) =
623 ps
Speedup = 623/819.81 = 0.76

4.15.3
The amount of LDUR and STUR instructions in the instruction mix will be the primary factor for distinguishing whether a program runs faster or slower on the new CPU. If a program depends heavily on LDUR and STUR instructions, then it will run slower as instruction count will double for LDUR/STUR. Programs that do not implement many LDUR/STUR instructions will have similar performance and runtime to when offset was used

4.15.4

I consider the original CPU as the better overall design. Combining two instructions into one for accessing memory makes the process much more efficient and quicker, as well as leading to better performance. This design can handle much more complicated applications at quicker runtimes. It seems like the new CPU can only be applied to simple implementations that use memory less frequently, and will obviously require more time to run with more instructions raising the CPI. Although the latency does drop, the tradeoff for performance loss is not worth it when running complex programs.

4.19

Initial: X1 = 11, X2 = 22

I am not sure if I have to add NOP operations. So I calculated it both ways!

ADDI X1, X2, #5
NOP
NOP
ADD X3, X1, X2
NOP
NOP
ADDI X4, X1, #15
NOP
NOP
ADD X5, X1, X1

ADDI X1, X2, #5

- X1 = X2 + 5 = 22 + 5 = 27 and no hazards

ADD X3, X1, X2

- X3 = X2 + X1 → X3 = 22 + 27 = 49
- Hazard: X1 is updated in the writeback stage of ADDI, so ADD must wait until the X1 value is written in.
- Therefore 2 NOPs needed after ADDI so that X1 is updated for next instruction.

ADDI X4, X1, #15

- X4 = X1 + 15 → X4 = 27 + 15 = 42
- Hazard- X1 is updated in the writeback stage of the previous instruction, so insert 2 NOPs after ADD to make sure X1 is updated.

ADD X5, X1, X1

- X5 = X1 + X1 → X5 = 27 + 27 = 54
- Hazard- X1 is updated in writeback stage of the previous instruction, so need 2 NOPS after ADDI to make sure X1 is updated.

With NOPs, the value of register X5 is 54

Without NOPs, the value of register X1 never updates accordingly and is 11 throughout. Therefore, the value of register X5 would be 11 + 11 = 22. X5 is 22.

4.20

ADDI X1, X2, #5

- X1 = X2 + 5
- Writes X1 in writeback, no dependencies/instructions before so no hazards.

ADD X3, X1, X2

- X3 = X1 + X2
- Hazard occurs here as X1 is written during the writeback of previous ADDI but does read X1 in CC3 before X1 is updated.
- Thus 2 NOPs are needed before this instruction to make sure that X1 updates in time.

ADDI X4, X1, #15

- X4 = X1 + 15
- Hazard: X1 is updated in CC5 during the writeback stage of the previous instruction. This current instruction reads X1 during CC4.
- Insert 2 NOPs before this instruction so that X1 is updated and has correct value.

ADD X5, X3, X2

- X5 = X3 + X2
- Hazard occurs as X3 is written in CC6 during the writeback stage of previous instruction. This current instruction reads it in CC6.
- Add one NOP before this instruction to ensure X1 has the correct value.

ADDI X1, X2, #5
NOP
NOP
ADD X3, X1, X2
NOP
NOP
ADDI X4, X1, #15
NOP
NOP
ADDI X4, X1, #15
NOP
ADD X5, X3, X2

4.22.1

5 stage pipeline where IF = instruction fetch, ID = instruction decode, EX = execution, MAC = memory access, WB = write back. Both instruction fetch and memory access use memory. GRAY = STALL.

	CC1	**CC2**	**CC3**	**CC4**	**CC5**	**CC6**	**CC7**	**CC8**	**CC9**	**CC10**	**CC11**	**CC12**	**CC13**
I1	IF	ID	EX	MAC	WB								
I2			IF	ID	EX	MAC	WB						
I3				IF	ID	EX	MAC	WB					
I4					IF	ID	EX	MAC	WB				
I5						IF	ID	EX	MAC	WB			
I6							IF	ID	EX	MAC	WB		

CC's are the clock cycles, I's are the instructions.

I1 = STUR X16, [X6, #12] | I2 = LDUR X16, [X6, #8] | I3 = SUB X7, X5, X4 | I4 = CBZ X7, Label

I5 = ADD X5, X1, X4 | I6 = SUB X5, X15, X4

Code stalls for one cycle before instruction 2 because there would be STUR in memory access and LDUR in instruction fetch at cycle 4 if a one-cycle stall did not exist.

4.22.2

If the code was reordered, then the stall could be eliminated or effectively hidden by first calling instruction 3 (SUB X7, X5, X4) first before then calling the LDUR instruction (I2). Since none of the registers involved in the SUB instruction appear in LDUR, this rearrangement is possible.

In general, reordering instructions/code can be possible, but it must be checked first that no other instructions are dependent on the moved instruction to stay in its ordered place. This includes register-use as well, as registers must keep their intended values for instructions. There also must be enough instructions to fill the stalls.

4.22.3

This structural hazard can be handled with NOPs, but it does come at a certain cost. Since there is a single memory, IF and MAC compete for the single memory and therefore cannot be in the same cycle when accessing memory for an instruction. Adding NOPs delays instructions without hardware changes, though it does increase the number of cycles required for the program. A NOP can be added before instruction 2 (LDUR) in this case. To avoid the increase in cycles though, there should be more than a single memory for the hardware to have less cycles.

4.22.4

Instruction Mix: R/I Type = 52%, LDUR = 25%, STUR = 10%, CBZ = 11%, B = 2%

This single memory structural hazard appears when handling memory-accessing instructions like LDUR and STUR in the same cycle regarding instruction-fetch and memory-access. LDUR and STUR need to access memory.

LDUR + STUR = 25 + 10 = 35%

Cannot know what percentage of instructions have LDUR and STUR consecutively, so use the worst case?

Worst Case = 35% of instructions would require stalls (or maybe 35/2?)

Therefore if a program has 100 total instructions, there would be 35 stalls in its worst case caused by the structural hazard.

4.25.1

Iteration 1:

```
LDUR X10, [X1, #0]  (I1)    // Load first pos on stack into X10, pop
LDUR X11, [X1, #8]  (I2)    // Load second pos on stack into X11, pop
ADD X12, X10, X11   (I3)    // Add X10 and X11 and assign to X12
SUBI X1, X1, #16    (I4)    // Subtract X1 by 16
CBNZ X12, LOOP      (I5)    // Compare X12 to 0, if it is not, then branch to loop
```

Iteration 2:

```
LDUR X10, [X1, #0]  (I6)
LDUR X11, [X1, #8]  (I7)
ADD X12, X10, X11   (I8)
SUBI X1, X1, #16    (I9)
CBNZ X12, LOOP      (I10)
```

5 stage pipeline where IF = instruction fetch, ID = instruction decode, EX = execution, MAC = memory access, WB = write back.

	CC1	**CC2**	**CC3**	**CC4**	**CC5**	**CC6**	**CC7**	**CC8**	**CC9**	**CC10**	**CC11**	**CC12**	**CC13**	**CC14**
I1	IF	ID	EX	MAC	WB									
I2		IF	ID	EX	MAC	WB								
I3			IF	ID	EX	MAC	WB							
I4				IF	ID	EX	MAC	WB						
I5					IF	ID	EX	MAC	WB					
I6						IF	ID	EX	MAC	WB				

I7							IF	ID	EX	MAC	WB			
I8								IF	ID	EX	MAC	WB		
I9									IF	ID	EX	MAC	WB	
I10										IF	ID	EX	MAC	WB

No stalls as there is perfect branch prediction and no stalls due to control hazards, along with no delay slots.

4.25.2

Start at IF stage of SUBI (CC4) and end at IF stage of CBNZ in second iteration (CC10)
So check cycles 4 to 10 to see if all five stages are processing useful instructions.
In cycle 4, there is no WB stage for any instruction, therefore it is not doing useful work.
In cycles 5-10 inclusive, all six stages of instructions are being performed so they are all doing useful work.

Therefore in the specified range, six out of the seven cycles are doing useful work.